DOSSIER PROJET

Application de gestion de commandes pour restaurant

Titre Professionnel Développeur Web et Web Mobile

Candidat	Germain Gildas AFOUBA
Organisme	CEF / préparation TP DWWM
Date	Avril 2026
Projet	Monorepo full-stack Node.js + React + Socket.io
Dépôt GitHub	https://github.com/ggildasafouba-spec/restaurant-order-system

Résumé exécutif

- Le projet digitalise la prise de commande, la préparation cuisine, le suivi client et la remise en salle.
- Le flux métier est visible en temps réel grâce à Socket.io.
- La démonstration montre clairement les compétences front-end et back-end du TP DWWM.

Livrables associés

- code source structuré en monorepo
- support de présentation de soutenance
- dossier projet détaillé et paginé
- dépôt GitHub public pour la remise

Objet du document. Ce dossier donne au jury une vue synthétique mais complète du besoin métier, des choix techniques, des compétences mobilisées, des tests réalisés et de la démarche suivie pour réaliser l'application.

REMERCIEMENTS ET SOMMAIRE

Remerciements. Je remercie les formateurs, la structure d'accueil, du stage et tout son personnel les ressources pédagogiques de la préparation TP DWWM et les retours de correction qui m'ont permis d'améliorer la qualité du dossier, de la structuration du projet et de la démonstration technique.

Sommaire

1. Introduction du projet 3
2. Front-end : maquettes, interfaces et expérience utilisateur 4
3. Front-end : responsive, accessibilité et SEO 5
4. Back-end : architecture, API et logique métier 6
5. Modèle de données, base de données et tests 7
6. Interconnexions, sécurité, RGPD et déploiement 8
7. Extraits de code et captures fonctionnelles 9
8. Difficultés rencontrées et conclusion 11

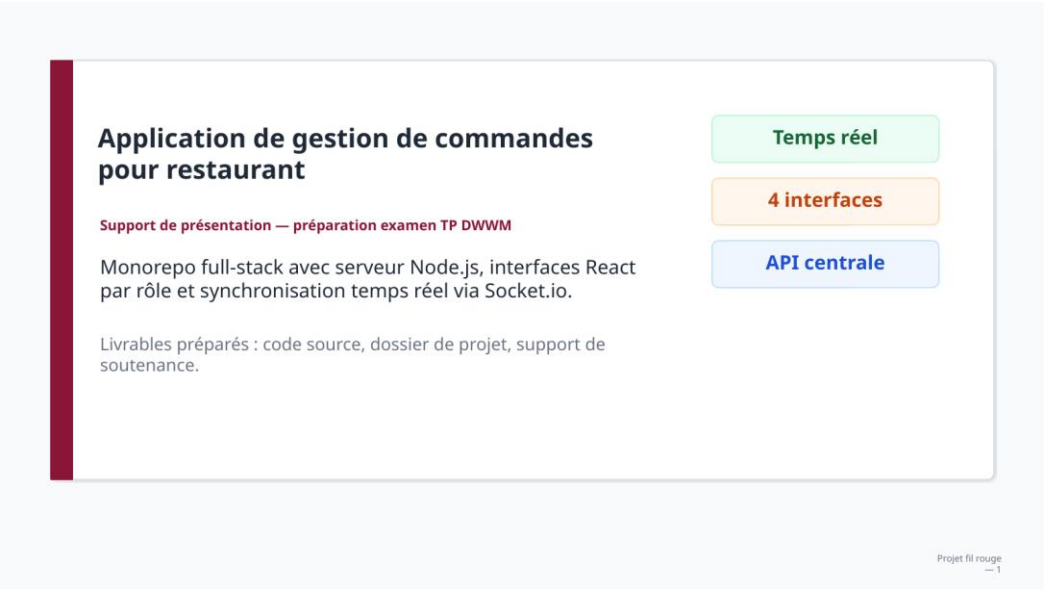

Figure 1 – Vue de synthèse du projet, reprise du support oral.

1. INTRODUCTION DU PROJET

Contexte et besoin métier. Dans un restaurant, les commandes passent souvent par plusieurs acteurs : la caisse, la cuisine, le service et le client. Le risque principal est la perte d'information, l'absence de visibilité sur l'avancement et la difficulté à synchroniser les équipes.

Objectif principal. Concevoir une application web capable de centraliser la création d'une commande, son envoi en cuisine, le suivi de son état et la confirmation finale de service.

Objectifs fonctionnels

- créer et payer une commande côté caisse
- faire évoluer son statut : new → in_preparation → ready → served
- afficher un suivi compréhensible pour le client
- permettre une démonstration claire pendant la soutenance

Valeur ajoutée

- réduction des erreurs de transmission
- visibilité instantanée pour chaque rôle
- processus métier compréhensible par le jury
- base exploitable pour une vraie application métier

Acteurs du système. Le rôle Client / Caisse crée la commande et valide le paiement simulé ; le rôle Cuisinier fait progresser l'état ; le rôle Serveur marque la commande comme servie ; l'écran Suivi client affiche la progression sans nécessiter d'accès à l'administration.

Figure 2 – Acteurs et parcours global de la commande.

2. FRONT-END : MAQUETTES, INTERFACES ET EXPÉRIENCE UTILISATEUR

Technologies côté client. Le front-end repose sur React avec Vite. Le stylage a été réalisé en CSS simple afin de conserver une interface rapide à comprendre, légère à maintenir et adaptée à une démonstration orale.

- React pour la composition d'interfaces par rôle métier.
- Vite pour le démarrage rapide et le découpage en applications indépendantes.
- CSS dédié par application pour contrôler la lisibilité et la cohérence visuelle.
- Socket.io client pour l'actualisation en temps réel des statuts.

Wireframe 1 – Écran Client / Caisse

En-tête : nom de l'écran + rôle affiché

Zone gauche : menu produits (nom, description, prix, bouton Ajouter)

Zone droite : type de commande, numéro de table, panier, total, validation

Zone basse : suivi visuel de la commande par étapes

Zoning fonctionnel utilisé pour cadrer l'interface avant codage.

Wireframe 2 – Écran Cuisinier

En-tête : rôle cuisine

Liste centrale : commandes en cours avec référence et détail des plats

Actions : bouton En préparation / bouton Prête

Actualisation automatique à chaque événement socket

Zoning fonctionnel utilisé pour cadrer l'interface avant codage.

Démarche de maquetage. Avant codage, le zoning a permis de séparer les zones critiques : sélection du menu, récapitulatif de commande, progression des statuts et commandes en attente. Cette étape a guidé le découpage des composants et l'ordre de développement.

3. FRONT-END : RESPONSIVE, ACCESSIBILITÉ ET SEO

Responsive et ergonomie

- La grille sépare les zones Menu et Commande sur grand écran pour accélérer la prise de commande.
- Le contenu reste exploitable sur écran plus étroit grâce à des blocs empilés et à des cartes simples.
- Les actions importantes (Ajouter, Payer et valider) restent visibles et explicites.

Accessibilité et bonnes pratiques

- présence d'étiquettes textuelles pour les champs de formulaire
- contraste renforcé sur les boutons d'action et les badges de rôle
- libellés explicites des étapes de progression
- choix d'une interface sobre pour réduire la charge cognitive

SEO et structure HTML. Même si l'application est surtout démontrée en local, la structure suit des principes SEO et sémantiques : un titre unique par vue, des sections identifiables, des intitulés de boutons explicites et une hiérarchie cohérente des titres. En cas de mise en ligne, des balises meta title et description seraient ajoutées dans chaque application Vite.

Exemple d'organisation sémantique d'une vue React

```
<header className="page-header">
  <h1>Client / Caisse</h1>
  <p className="small">Prise de commande, paiement simulé et suivi temps réel.</p>
</header>
<section className="card">...</section>
```

Extrait de code commenté pour la soutenance.

Figure 3 – Démonstration fonctionnelle du flux métier, utile pour l'oral.

4. BACK-END : ARCHITECTURE, API ET LOGIQUE MÉTIER

Technologies serveur. Le back-end utilise Node.js, Express et Socket.io. Cette pile permet d'exposer une API REST simple et d'émettre des événements temps réel lors de la création ou de la progression d'une commande.

- Express gère les routes `/api/menu`, `/api/orders` et `/api/orders/:ref/status`.
- Socket.io diffuse `order:new`, `order:update`, `order:ready`, `order:served` et `order:progress`.
- La logique métier est concentrée dans les contrôleurs et le store mémoire, ce qui rend le flux lisible pendant la soutenance.

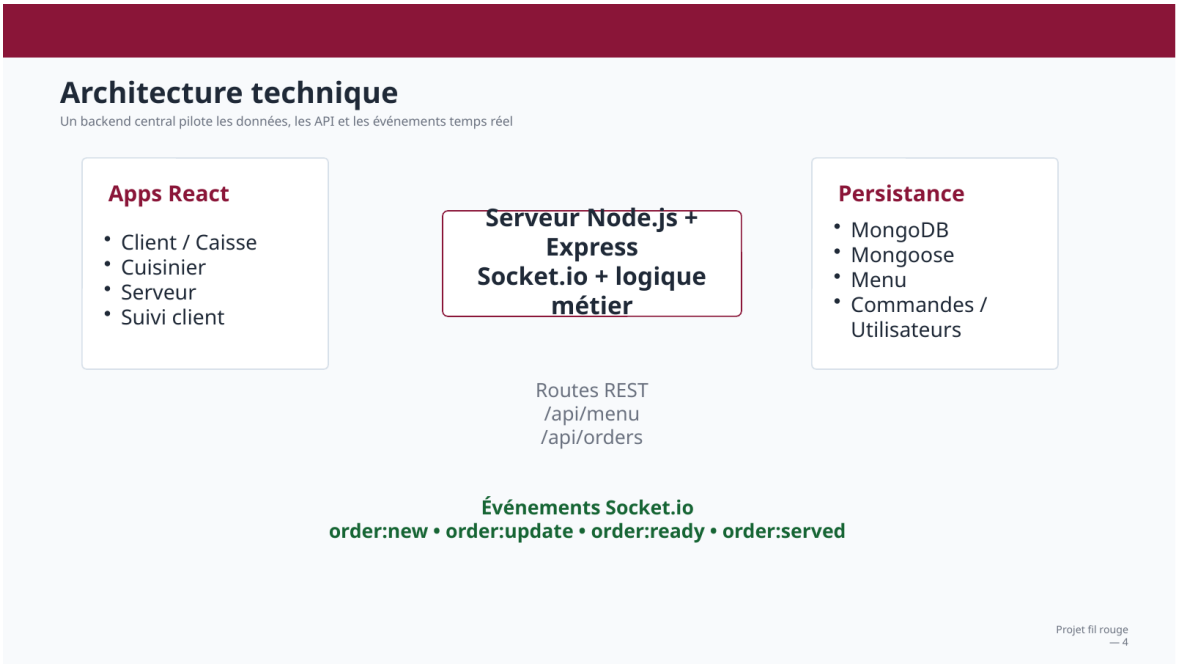

Figure 4 – Architecture technique générale du projet.

Architecture logique. Le projet suit une séparation claire : routes → contrôleurs → store / modèles. Cette organisation s'apparente à un MVC léger : les vues sont dans les applications React, les contrôleurs traitent les requêtes HTTP et le store représente la couche de données de démonstration.

Extrait du serveur principal

```
registerMenuRoutes(app);
registerOrderRoutes(app, io);
attachSocketHandlers(io);
const PORT = process.env.PORT || 4000;
connectDb().finally(() => server.listen(PORT));
```

Extrait de code commenté pour la soutenance.

5. MODÈLE DE DONNÉES, BASE DE DONNÉES ET TESTS

Modèle de données métier (vue logique)

Entité	Champs principaux	Relations	État dans la version livrée
MenuItem	id, name, description, price, category	référencé par OrderItem.menuItemId	utilisé pour construire le menu côté caisse
Order	id, ref, orderType, tableNumber, total, paid, status, createdAt, updatedAt	1 commande contient n lignes d'items	stocké en mémoire via memoryStore, exposé par API
OrderItem	menuItemId, name, quantity, unitPrice	n lignes rattachées à 1 Order	intégré au document de commande
User (cible)	id, name, role	rôle : caisse / cuisine / service	modèle préparé pour une évolution ultérieure

Le code de démonstration fonctionne aujourd'hui avec un magasin mémoire ; la configuration MongoDB est prête pour une persistance complète.

Base de données et persistance. Dans la version présentée au jury, les commandes sont stockées dans `memoryStore`, ce qui garantit une démonstration simple, rapide et sans dépendance bloquante. Le projet intègre cependant une configuration MongoDB et des modèles métiers (`MenuItem`, `Order`, `User`) afin de préparer une persistance plus complète.

Jeux de tests réalisés. Les tests ont été principalement menés sous forme de tests fonctionnels manuels et de vérifications d'API comparables à un scénario Postman. L'objectif était de vérifier la cohérence du flux complet et la bonne émission des événements temps réel.

Cas testé	Méthode / route	Résultat attendu	Résultat observé
Récupération du menu	GET /api/menu	liste des produits	OK
Création de commande	POST /api/orders	commande créée avec ref A00x	OK
Passage en préparation	PATCH /api/orders/:ref/status	statut `in_preparation` et event update	OK
Commande prête	PATCH /api/orders/:ref/status	statut `ready` et event ready	OK
Commande servie	PATCH /api/orders/:ref/status	statut `served` et event served	OK

Tableau de tests fonctionnels utilisé pour documenter la démonstration.

6. INTERCONNEXIONS, SÉCURITÉ, RGPD ET DÉPLOIEMENT

Lien front / back. Chaque application React consomme l'API centrale via la fonction partagée `fetchJson`. Les vues se connectent aussi au serveur Socket.io pour recevoir les changements de statut sans rechargement de page. Ce couplage maîtrisé constitue le cœur de l'expérience temps réel.

Extrait du module partagé API

```
const API_URL = import.meta.env.VITE_API_URL || 'http://localhost:4000';
export async function fetchJson(path, options = {}) {
  const response = await fetch(`${API_URL}${path}`, {...options});
  if (!response.ok) throw new Error(`Erreur API: ${response.status}`);
  return response.json();
}
```

Extrait de code commenté pour la soutenance.

Sécurité et protections majeures

- validation des statuts autorisés côté serveur avant mise à jour
- séparation des rôles par interfaces dédiées pour limiter les erreurs de manipulation
- préparation à la centralisation des règles métier dans les contrôleurs
- sensibilisation aux risques XSS, CSRF et injections dans la phase d'évolution

RGPD et confidentialité

- aucune donnée sensible de paiement réel n'est stockée dans la démonstration
- limitation des données collectées au strict nécessaire : table, contenu de commande, état
- recommandation d'ajouter une politique de conservation et de journalisation en production
- sécurisation future des accès par authentification et rôles serveur

Gestion de version et dépôt en ligne. Le projet est versionné avec Git et publié sur GitHub afin de faciliter la remise et la relecture : https://github.com/ggildasafouba-spec/restaurant-order-system

Déploiement / hébergement. La démonstration est aujourd'hui prévue en environnement local (`npm install`, copie de `.env`, `npm run dev`). Une mise en ligne peut être envisagée avec un hébergement Node.js côté API (Render, Railway) et un front Vite déployé séparément. Cette étape n'a pas encore été industrialisée, mais l'architecture du projet s'y prête.

7. EXTRAITS DE CODE ET CAPTURES FONCTIONNELLES

Extrait front-end : ajout au panier et validation. La logique suivante montre comment l'écran caisse alimente le panier puis envoie la commande au back-end.

```
function addToCart(item) {
  setCart((cur) => {
    const e = cur.find((x) => x.id === item.id);
    return e ? cur.map((x) => x.id === item.id ? { ...x, quantity: x.quantity + 1 } : x)
             : [...cur, { ...item, quantity: 1 }];
  });
}
async function submitOrder() { /* POST /api/orders */ }
```

Extrait de code commenté pour la soutenance.

Extrait back-end : progression de commande. Le contrôleur centralise la mise à jour et émet les notifications Socket.io appropriées.

```
if (!Object.values(ORDER_STATUS).includes(status)) return res.status(400).json(...);
const order = memoryStore.updateStatus(req.params.ref, status);
if (status === ORDER_STATUS.READY) emitProgress(io, order, 'order:ready');
else if (status === ORDER_STATUS.SERVED) emitProgress(io, order, 'order:served');
else emitProgress(io, order);
```

Extrait de code commenté pour la soutenance.

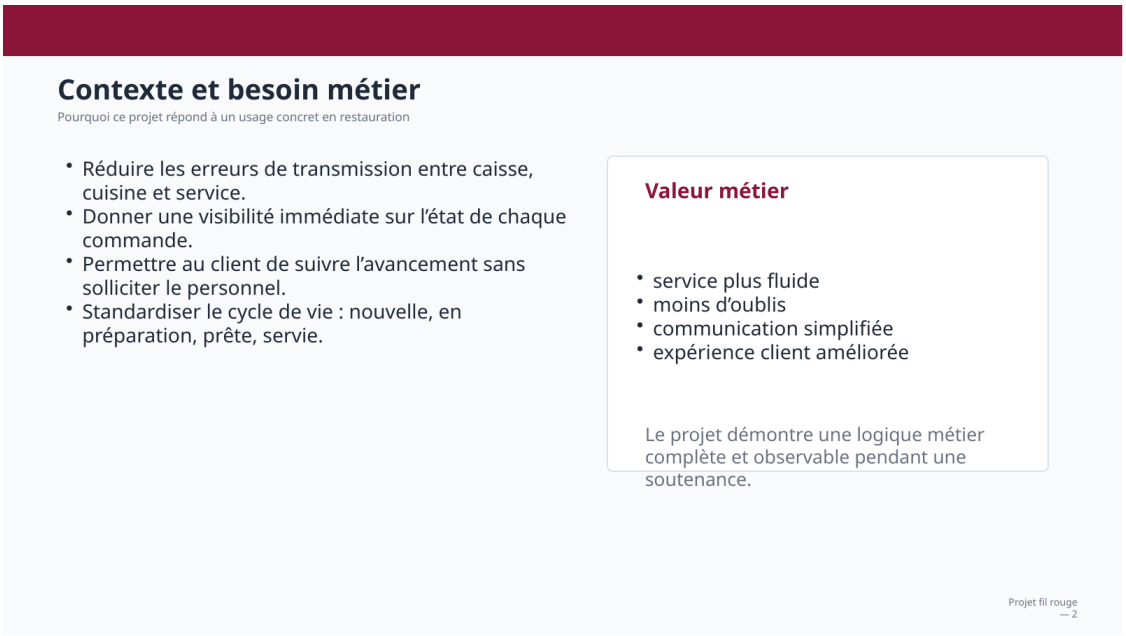

Figure 5 – Rappel du besoin métier et de la valeur apportée.

8. DIFFICULTÉS RENCONTRÉES ET CONCLUSION

Difficultés rencontrées. La première difficulté a été de synchroniser plusieurs interfaces sans perdre la lisibilité du code. La seconde concernait la structuration du dossier projet : le premier rendu était trop léger et n'explicitait pas assez les maquettes, le modèle de données, les tests et les aspects transverses (RGPD, sécurité, déploiement).

- Correction de la structure du dossier avec page de garde, remerciements, sommaire et pagination.
- Ajout d'un zoning fonctionnel et d'une explication des interfaces front-end.
- Ajout d'une description détaillée du back-end, du modèle de données et des tests.
- Formalisation des aspects transverses : Git/GitHub, sécurité, RGPD et déploiement local.

Bilan personnel. Ce projet m'a permis de démontrer une chaîne complète de développement web et web mobile : réflexion métier, découpage front-end, API REST, actualisation temps réel, organisation du code source et documentation de soutenance. Il constitue une base crédible pour une application de restauration plus aboutie.

Conclusion générale. L'application répond au besoin de fluidifier la prise de commande et le suivi en restaurant. Elle met en évidence les compétences front-end, back-end et transverses du TP DWWM. Les prochaines évolutions prioritaires seraient la persistance complète en base de données, l'authentification par rôle, la journalisation et le déploiement cloud.

Perspectives

- authentification par rôle
- tableau de bord d'administration
- impression ticket
- historique et statistiques

Message final

- Le dossier est désormais structuré, argumenté et aligné sur les critères de correction.
- Le support peut être utilisé tel quel pour la soutenance puis enrichi selon les retours du jury.